Modélisation, Modularité et Architecture Logicielle en C++

Thème principal : Conception modulaire et architecture logicielle

Synthèse exécutive : Ce cours de synthèse du module R1.05 formalise les principes cardinaux de la conception logicielle et de la modélisation en BUT Informatique à l'IUT de Bayonne et du Pays Basque. L'objectif est d'apprendre à découper un problème complexe en sous-modules indépendants à responsabilité unique, à spécifier rigoureusement les flux de données (entrées/sorties), à éliminer les effets de bord et à proscrire toute dépendance circulaire afin d'écrire des programmes robustes, maintenables et testables.

Fondations de la Modélisation et Conception Modulaire

En informatique moderne, il est proscrit de concevoir une application sous la forme d'un bloc monolithique d'instructions. Un problème global doit être analysé puis subdivisé en une série de sous-problèmes élémentaires indépendants. Chaque sous-problème est pris en charge par un module fonctionnel précis. Cette démarche de modélisation réduit la charge cognitive lors de la conception, facilite la localisation rapide des bogues, permet la répartition du travail au sein d'une équipe informatique et garantit la réutilisabilité du code sans devoir recopier des séquences d'instructions.

- Modularité : Décomposition d'un système informatique complexe en blocs fonctionnels autonomes, réutilisables et testables séparément.
- Responsabilité Unique : Principe selon lequel chaque sous-programme ou module ne doit exécuter qu'une seule tâche parfaitement définie.
- Maintenabilité et Évolutivité : Aptitude d'un code à subir des modifications, corrections ou extensions sans risquer d'altérer d'autres parties du système.

Cas concret enseignant : *Le professeur compare l'architecture logicielle à la construction d'une charpente en menuiserie : si les fondations conceptuelles ne sont pas alignées et si les liaisons entre les poutres sont mal calculées, le moindre ajout ultérieur ou le moindre changement fait écrouler l'ensemble de la structure.*

Démarche Logique et Analyse des Flux de Données

La résolution d'un problème algorithmique ne débute jamais par la saisie du code dans l'IDE. Elle démarre par une phase explicite de spécification logicielle décomposée en trois temps : 1) Répertorier l'ensemble des données d'entrée fournies par l'utilisateur ou l'environnement système, ainsi que la nature des sorties attendues. 2) Fractionner la logique algorithmique globale en traitements unitaires indépendants. 3) Définir formellement les prototypes de fonctions associés, en s'assurant qu'aucune variable globale n'est exploitée afin de préserver l'étanchéité des modules.

- Analyse préalablement au codage : Étape fondamentale consistant a isoler formellement les paramètres d'entrée, les traitements et les valeurs d'échappement/sortie.
- Isolation fonctionnelle : Garantie qu'un sous-programme ne manipule et ne modifie que les variables explicitement reçues par paramètre.
- Contrat d'interface : Définition exacte du type, du rôle et du sens d'échange des paramètres entre le programme appelant et le module appelé.

Cas concret enseignant : *Cas algorithmique standard : la séparation stricte entre l'acquisition d'un ensemble de notes saisies au clavier (avec lecture de sentinelle -1), le sous-programme de calcul arithmétique de la moyenne, et le sous-programme d'affichage des résultats pour l'utilisateur.*

Prévention des Pièges d'Architecture et Effets de Bord

L'apparition de dépendances circulaires représente une erreur majeure de conception logicielle. Pour éviter que deux modules ne deviennent prisonniers l'un de l'autre, la structure logicielle doit obéir à une hiérarchie stricte orientée de haut en bas : les modules de haut niveau orchestrent la logique métier en faisant appel aux modules de bas niveau (modules utilitaires ou de calcul), sans que ces derniers n'aient jamais connaissance des modules appelants. De surcroît, le passage rigoureux de paramètres élimine tout risque d'effet de bord.

- Dépendance Circulaire : Situation critique où deux modules dépendent mutuellement l'un de l'autre (A appelle B et B appelle A), créant des blocages d'interdépendance.
- Effet de bord : Altération non souhaitée, masquée ou imprévisible de l'état global de la mémoire ou de variables externes lors de l'exécution d'un sous-programme.
- Couplage faible : Objectif d'architecture consistant à réduire au strict minimum le niveau d'interdépendance entre les différents modules d'une application.

Cas concret enseignant : *Le professeur insiste sur le fait qu'une fonction de calcul pure ne doit JAMAIS effectuer de 'cout' ou de 'cin' direct en son sein, sous peine de détruire son autonomie et de la coupler définitivement avec la console.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce que cette synthèse sur la modularité sera directement évaluée sous forme de question de cours en partiel ?
Réponse du professeur : Oui, la maîtrise des concepts de couplage, de dépendance circulaire et de responsabilité unique fait systématiquement l'objet de questions d'analyse ou de points de notation stricts lors des évaluations écrites.

■ Exercice & Démonstration d'Application

Énoncé : Concevoir et implémenter un programme C++ modulaire conforme à la norme de l'IUT d'Anglet permettant de calculer la moyenne d'une série de valeurs positives. Le calcul de la moyenne doit être complètement isolé dans un sous-programme respectant le principe de responsabilité unique (sans entrées/sorties console dans la fonction de calcul).

Démarche de résolution :
1. Identification des entrées/sorties : la fonction main gère les interactions console (cin, cout) tandis que la fonction calculerMoyenne reçoit les valeurs par paramètres explicites. 2. Application stricte du style Allman pour les accolades et de la tripartition (CONSTANTES, VARIABLES, TRAITEMENTS) dans chaque bloc exécutable. 3. Intégration des commentaires de flux de données (clavier >> saisir >> variable) conformes aux exigences de l'IUT d'Anglet. 4. Validation préventive du dénominateur (nombreValeurs > 0) avant l'appel pour éviter toute division par zéro.

```cpp
/**
 * Fichiers : calculMoyenne.cpp
 * Auteur : Matéi
 * Cours : R1.05 - IUT Anglet
 */

#include <iostream>

using namespace std;

// CONSTANTES
const int SENTINELLE = -1;

// PROTOTYPES
double calculerMoyenne(double pSomme, int pNbValeurs);

int main()
{
    // CONSTANTES
    // VARIABLES
    double noteCourante;
    double sommeNotes;
    int nombreNotes;
    double moyenne;

    // TRAITEMENTS
    sommeNotes = 0.0;
    nombreNotes = 0;
    moyenne = 0.0;

    cout << "Entrez une note (ou -1 pour arreter) : ";
    // (clavier) >> saisir >> noteCourante
    cin >> noteCourante;

    while (noteCourante != SENTINELLE)
    {
        // TRAITEMENTS
        sommeNotes = sommeNotes + noteCourante;
        nombreNotes = nombreNotes + 1;

        cout << "Entrez une note (ou -1 pour arreter) : ";
        // (clavier) >> saisir >> noteCourante
        cin >> noteCourante;
    }

    if (nombreNotes > 0)
    {
        // (variables) >> appeler >> calculerMoyenne
        moyenne = calculerMoyenne(sommeNotes, nombreNotes);
        cout << "La moyenne est de : " << moyenne << endl;
    }
    else
    {
        cout << "Aucune note saisie." << endl;
    }

    return 0;
}

double calculerMoyenne(double pSomme, int pNbValeurs)
{
    // CONSTANTES
    // VARIABLES
    double resultat;

    // TRAITEMENTS
    resultat = pSomme / pNbValeurs;

    return resultat;
}
```

Résultat : Un code C++ modulaire, maintenable et autonome, où la logique de calcul est parfaitement disjointe de la logique de présentation console, garantissant la réutilisabilité du composant.

■ Auto-Évaluation Flash d'Examen

Q : Pourquoi est-il interdit d'effectuer des saisies (cin) ou des affichages (cout) au sein d'un sous-programme purement calculatoire ?

R : Parce que cela crée un couplage fort avec la console, détruit l'autonomie de la fonction et empêche sa réutilisation dans un autre contexte (par exemple, une interface graphique ou un test automatisé).

Q : Qu'est-ce qu'une dépendance circulaire et comment la prévenir ?

R : C'est une situation où deux modules dépendent mutuellement l'un de l'autre. On la prévient en instaurant une hiérarchie de haut en bas claire où les modules de haut niveau appellent ceux de bas niveau sans retour inverse.

Q : Quelles sont les trois sections obligatoires de la tripartition exigée dans les codes C++ à l'IUT d'Anglet ?

R : CONSTANTES, VARIABLES et TRAITEMENTS.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765